UNIVERSIDADE FEDERAL DE SERGIPE
DISCIPLINA DE SISTEMAS DISTRIBUÍDOS

Projeto de Sistemas Distribuídos

ALUNO				**MATRÍCULA**	
DISCIPLINA	Sistemas Distribuídos			**DATA DA PROVA**	21/10/2026
PROFESSOR	Rafael Oliveira Vasconcelos			**TIPO DE PROVA**	Projeto P1
TURMA		**CÓDIGO DA TURMA**	COMP0470	**NOTA**	

ATENÇÃO:
- Ao aluno flagrado **utilizando meios ilícitos ou não autorizado pelo professor para responder a avaliação** será atribuída nota zero e, mediante representação do professor, responderá a Procedimento Administrativo Disciplinar, com base no Código de Ética.
- São critérios de avaliação a qualidade e eficiência da implementação, indentação e legibilidade do código, e uso dos conceitos aprendidos na unidade. A implementação deve estar adequada para o seu uso.
- O entendimento e interpretação das questões fazem parte da avaliação;
- Os quesitos que não possam ser executados (sem compilar) serão desconsiderados durante a correção.
- Resolução com má legibilidade ou fora da especificação valerá no máximo 50% da pontuação da questão.
- A resolução da atividade deverá ser **enviada via GitHub**. Envios realizados após o horário da atividade ou que não sejam postados na tarefa referente à atividade serão DESCONSIDERADOS. A responsabilidade do envio dos arquivos é do(a) aluno(a). Confira o arquivo que foi enviado.

1. Objetivo do projeto

O projeto consiste no desenvolvimento de um **Simulador de Bolsa de Valores Multiplayer**, no qual diversos jogadores participam simultaneamente de um mercado financeiro virtual.

Cada jogador assume o papel de um investidor e recebe um **capital virtual inicial** para realizar operações de compra e venda de ativos. O sistema deverá simular o funcionamento básico de uma bolsa de valores, permitindo que vários investidores enviem ordens simultaneamente e observem, em tempo real, as alterações no mercado.

O objetivo não é reproduzir integralmente uma bolsa de valores real, mas construir uma **simulação distribuída de um mercado financeiro**, permitindo aos estudantes explorar problemas clássicos de Sistemas Distribuídos.

O projeto deverá envolver situações nas quais diversos clientes realizam operações concorrentes sobre um mesmo conjunto de dados. Dessa forma, problemas como **consistência, concorrência, ordenação de eventos, disponibilidade, tolerância a falhas e sincronização de estado** tornam-se elementos centrais do sistema.

Cada grupo poderá definir detalhes do mercado, como:

- quantidade e tipos de ativos;
- quantidade de jogadores;
- capital inicial;
- regras de negociação;
- horário de funcionamento;
- eventos econômicos;



- mecanismos de formação de preços;
- estratégias automatizadas;
- ranking dos investidores.

2. Cenário

Vários investidores estarão conectados simultaneamente ao sistema. Cada um utilizará um Home Broker para consultar o mercado, visualizar seu patrimônio e enviar ordens. O servidor ou conjunto de serviços será responsável por validar as operações, manter o estado do mercado, processar as ordens e informar as mudanças aos clientes.

Uma única negociação poderá alterar simultaneamente o saldo, a carteira, o livro de ofertas, o histórico de operações e as informações exibidas para diversos investidores. Esse comportamento constitui o principal cenário para exploração dos problemas de sistemas distribuídos.

3. Conceitos básicos: Bolsa de Valores

Uma bolsa de valores é um ambiente organizado no qual compradores e vendedores podem negociar ativos financeiros.

No contexto simplificado deste projeto, existirão três elementos principais:

Investidores → envio de ordens → Mercado → execução das negociações

Um investidor pode, por exemplo, desejar comprar:

- 100 ações da empresa ABC por até R$ 25,00 cada.

Outro investidor pode estar disposto a vender:

- 100 ações da empresa ABC por R$ 25,00 cada.

Quando as condições das ordens são compatíveis, o sistema pode realizar uma **negociação (trade)**.

O preço do ativo poderá variar de acordo com as ordens existentes no mercado e com as negociações realizadas.

Exemplo

Suponha que o livro de ofertas contenha:

Venda	Quantidade	Preço
Investidor A	100	R$ 102

Venda	Quantidade	Preço
Investidor B	200	R$ 103
Investidor C	300	R$ 105

E as ordens de compra:

Compra	Quantidade	Preço
Investidor D	150	R$ 101
Investidor E	100	R$ 100
Investidor F	200	R$ 98

O melhor preço de venda é **R$ 102**, enquanto o melhor preço de compra é **R$ 101**.

Nesse momento não existe uma negociação, pois nenhum comprador está disposto a pagar o preço solicitado pelo vendedor.

Se o Investidor D alterar sua ordem para:

Comprar 150 ações por até R$ 102.

o sistema deverá realizar o casamento das ordens, executando a negociação conforme as regras definidas pelo grupo.

Esse mecanismo será uma das partes mais importantes do projeto.

4. Livro de ofertas e Matching Engine

Cada grupo deverá implementar um **motor de negociação (matching engine)**.

Esse componente será responsável por receber as ordens dos investidores e determinar quando elas podem ser executadas.

Uma possibilidade é utilizar o princípio:

Prioridade de preço + prioridade temporal.

Por exemplo, para compras:

1. maior preço tem prioridade;
2. em caso de empate, a ordem mais antiga tem prioridade.

Para vendas:



1. menor preço tem prioridade;
2. em caso de empate, a ordem mais antiga tem prioridade.

Isso cria um problema interessante para Sistemas Distribuídos:

O que acontece quando dois clientes enviam ordens praticamente ao mesmo tempo?

Uma política possível é utilizar prioridade de preço e, em caso de empate, prioridade temporal. Para compras, preços maiores têm prioridade; para vendas, preços menores têm prioridade. O grupo poderá adotar outra política, desde que seja claramente especificada e tecnicamente justificada.

VENDA	Preço	COMPRA	Preço
100 ações	R$ 105,00	150 ações	R$ 102,00
200 ações	R$ 106,00	300 ações	R$ 101,00
500 ações	R$ 108,00	200 ações	R$ 99,00

5. Home Broker

O **Home Broker** será a interface utilizada pelo jogador para participar do mercado.

Em uma aplicação real, o investidor utiliza o Home Broker para visualizar informações do mercado e enviar ordens.

No projeto, cada jogador deverá possuir uma interface semelhante a um Home Broker simplificado.

5.1 Informações mínimas

- Saldo disponível.
- Patrimônio total.
- Ativos possuídos.
- Quantidade e preço médio de cada ativo.
- Lucro/prejuízo da carteira.
- Lista de ativos negociáveis.
- Preço atual e variação dos ativos.
- Volume negociado.
- Melhores ofertas de compra e venda.
- Histórico de ordens e negociações.

5.2 Operações mínimas

- Enviar ordem de compra.
- Enviar ordem de venda.
- Consultar o status de uma ordem.
- Consultar a carteira.
- Consultar o histórico de negociações.



5.3 Atualização em tempo real

O Home Broker deverá receber atualizações quando ocorrerem eventos relevantes, como execução de ordens, alteração do livro de ofertas, mudança de preço, rejeição de ordem, desconexão e reconexão.

6. Exemplo de funcionamento multiplayer

Considere três investidores conectados simultaneamente. João envia uma ordem de compra, Maria envia outra ordem de compra e Pedro envia uma ordem de venda. O sistema deverá receber, validar, ordenar e processar essas operações de acordo com as regras do mercado.

1. Receber a ordem.
2. Validar saldo ou quantidade de ativos.
3. Inserir ou casar a ordem no livro de ofertas.
4. Executar a negociação, quando houver compatibilidade.
5. Atualizar saldo e carteira.
6. Atualizar o livro de ofertas e informações de mercado.
7. Persistir os dados relevantes.
8. Notificar os clientes afetados.

7. Requisitos técnicos

7.1 Arquitetura distribuída

- Utilizar arquitetura cliente-servidor ou outra arquitetura distribuída devidamente justificada.
- O servidor ou conjunto de serviços deverá controlar a lógica principal do mercado.
- O sistema deverá sincronizar o estado observado pelos clientes.
- Sempre que possível, a solução deverá adotar serviços/microsserviços.
- O sistema deverá ser executado em contêineres e, quando tecnicamente aplicável, em um cluster.
- A arquitetura deverá ser documentada por meio de diagramas.

7.2 Comunicação em rede

Poderão ser utilizados TCP, HTTP/REST, WebSocket, UDP, gRPC, MQTT, AMQP ou outros protocolos, desde que justificados.

- Comunicação efetiva entre cliente e servidor/serviços.
- Troca eficiente de ordens e eventos.
- Atualização do mercado em tempo real.
- Tratamento de erros de comunicação.

7.3 Consistência e concorrência

- Tratar múltiplas ordens simultâneas.
- Evitar condições de corrida.
- Impedir que o mesmo saldo ou ativo seja utilizado indevidamente em operações concorrentes.
- Definir política de ordenação das ordens.
- Definir o modelo de consistência utilizado em cada parte relevante.
- Explicar como são garantidas atomicidade e integridade das negociações.



O grupo deverá discutir explicitamente as decisões relacionadas a consistência forte ou eventual, quando aplicáveis, e justificar suas escolhas.

7.4 Gerenciamento de sessões

- Cadastro/login e autenticação.
- Entrada e saída do mercado/sala.
- Gerenciamento de sessão.
- Identificação dos investidores conectados.
- Reconhecimento do usuário após reconexão.

7.5 Tolerância a falhas

- Detectar desconexão de clientes.
- Permitir reconexão.
- Recuperar o estado do investidor quando possível.
- Definir o comportamento de ordens quando ocorre uma falha.
- Considerar falha de serviço e/ou indisponibilidade temporária de componentes.
- Demonstrar pelo menos uma falha controlada durante a apresentação final.

7.6 Persistência de dados

- Usuários.
- Saldo.
- Carteira.
- Ativos.
- Ordens.
- Negociações.
- Histórico de operações.

O grupo deverá justificar a tecnologia escolhida para persistência e explicar como trata concorrência, consistência e recuperação. O uso de banco distribuído é recomendado.

7.7 Escalabilidade e testes de carga

O sistema deverá ser submetido a testes com diferentes quantidades de usuários simultâneos. Sugestão:

Cenário	Usuários simultâneos
Teste 1	5
Teste 2	20
Teste 3	50
Teste 4	100 ou mais

- Latência.
- Throughput.
- Ordens processadas por segundo.
- Tempo de execução de negociações.



- CPU e memória.
- Tempo de resposta.
- Comportamento sob sobrecarga.

7.8 Observabilidade

- Logs estruturados ou adequadamente organizados.
- Métricas relevantes.
- Registro de erros.
- Monitoramento dos serviços.
- Possibilidade de acompanhar uma ordem desde sua entrada até a execução.

Exemplo de fluxo observável: ORDER_RECEIVED → ORDER_VALIDATED → ORDER_BOOK_INSERTED → MATCH_FOUND → TRADE_EXECUTED → PORTFOLIO_UPDATED → CLIENTS_NOTIFIED.

7.9 Segurança

- Autenticação.
- Autorização.
- Validação de mensagens e parâmetros.
- Proteção das operações dos usuários.
- Controle de acesso aos recursos.
- Impedir alteração indevida de saldo ou carteira.
- Impedir cancelamento ou alteração de ordens pertencentes a outro usuário.

8. Simulação do mercado e IA

Para que o mercado continue ativo mesmo quando houver poucos alunos conectados, o sistema deverá possuir agentes automatizados (NPCs/investidores virtuais).

Os agentes poderão:

- Comprar e vender ativos.
- Reagir a alterações de preço.
- Reagir a eventos econômicos.
- Seguir estratégias de investimento.
- Possuir perfis de risco diferentes.
- Tomar decisões baseadas em regras ou algoritmos.

Não é obrigatório utilizar Machine Learning. Um algoritmo baseado em regras também poderá ser utilizado, desde que seja relevante para a simulação e esteja documentado.

9. Eventos econômicos

O grupo poderá implementar eventos que alterem o comportamento do mercado, como notícias positivas ou negativas, mudanças econômicas ou eventos específicos das empresas virtuais.

- Alta ou queda inesperada de uma empresa.
- Alteração na demanda por determinado ativo.



- Evento econômico geral.
- Notícia que altera a estratégia dos agentes.

Os grupos deverão explicar como os eventos são gerados, processados e propagados entre os componentes do sistema.

10. Requisitos mínimos obrigatórios

ID	Requisito	Descrição
R1	Múltiplos clientes	Pelo menos dois investidores devem poder operar simultaneamente.
R2	Autenticação	Cada jogador deve possuir uma sessão identificável.
R3	Home Broker	Interface para consultar mercado e carteira e enviar ordens.
R4	Ativos	Existência de ativos virtuais negociáveis.
R5	Ordens	Compra e venda com validação.
R6	Livro de ofertas	Manutenção das ordens abertas.
R7	Matching Engine	Processamento e execução de ordens compatíveis.
R8	Sincronização	Atualização consistente das informações dos clientes.
R9	Persistência	Armazenamento de operações e estado relevante.
R10	IA/agentes	Investidores automatizados participando do mercado.
R11	Falhas	Reconexão e/ou recuperação de estado.
R12	Escalabilidade	Testes com diferentes cargas.
R13	Observabilidade	Logs e métricas suficientes para analisar o sistema.

R14	Segurança	Autenticação, autorização e validação das operações.
R15	Contêineres	Execução dos componentes em contêineres.

11. Requisitos avançados e diferenciais

- Replicação de serviços.
- Particionamento/sharding.
- Filas ou streaming de eventos.
- Circuit breaker.
- Descoberta de serviços.
- Balanceamento de carga.
- Eleição de líder.
- Mecanismos de consenso.
- Event sourcing.
- CQRS (Command Query Responsibility Segregation, ou Separação de Responsabilidade de Consulta e Comando).
- Múltiplos mercados independentes.
- Agentes com estratégias distintas.
- Dashboard avançado de observabilidade.
- Testes automatizados de falhas/caos.
- Simulação de milhares de investidores.

Esses itens não são obrigatórios, mas poderão ser considerados diferenciais quando implementados de forma correta e tecnicamente justificada.

12. Etapas de desenvolvimento

12.1 Parte 1 — Proposta

- Definição das regras do mercado.
- Descrição dos ativos e investidores.
- Arquitetura proposta.
- Diagrama dos componentes.
- Tecnologias escolhidas.
- Protocolos de comunicação.
- Banco de dados.
- Estratégia de consistência.
- Estratégia de tolerância a falhas.
- Proposta dos agentes/IA.
- Cronograma de desenvolvimento.

12.2 Parte 1 — Protótipo

- Conexão de múltiplos clientes.
- Autenticação.
- Entrada no mercado.
- Visualização dos ativos.
- Envio de ordens.
- Comunicação cliente-servidor.
- Atualização básica do Home Broker.

12.3 Parte 2 — Implementação avançada

- Matching Engine.
- Livro de ofertas.
- Execução de negociações.
- Carteira e patrimônio.
- Persistência.
- Sincronização.
- IA/agentes.
- Tolerância a falhas.
- Reconexão.
- Escalabilidade.
- Observabilidade.
- Segurança.

12.4 Parte 2 — Teste e validação

- Testes funcionais.
- Testes de concorrência.
- Testes de carga.
- Testes de falhas.
- Avaliação de recuperação.
- Medição de desempenho.
- Análise dos resultados.

12.5 Entrega final

- Código-fonte completo.
- Documentação técnica.
- Instruções de execução.
- Arquivos de configuração.
- Relatório de testes.
- Demonstração prática.

13. Critérios de avaliação — Parte 1

Categoria	Critério	Peso
Funcionalidade e Criatividade	Funcionamento do protótipo	14%

Funcionalidade e Criatividade	Criatividade e inovação do mercado	14%
Funcionalidade e Criatividade	Escolha e utilização das tecnologias	14%
Funcionalidade e Criatividade	Segurança, escalabilidade e tolerância a falhas	14%
Funcionalidade e Criatividade	Cronograma, andamento e acompanhamento	14%
Documentação e Apresentação	Documentação técnica	15%
Documentação e Apresentação	Apresentação e demonstração	15%

Total: 100%.

14. Critérios de avaliação — Parte 2

Categoria	Critério	Peso
Aspectos Técnicos	Arquitetura distribuída e modularização	15%
Aspectos Técnicos	Comunicação e sincronização	15%
Aspectos Técnicos	Consistência, concorrência e tolerância a falhas	15%
Aspectos Técnicos	Persistência, escalabilidade, segurança e observabilidade	15%
Funcionalidade e Criatividade	Funcionamento geral do simulador	10%
Funcionalidade e Criatividade	Criatividade, mecânicas e diferenciais	10%
Documentação e Apresentação	Documentação técnica	10%
Documentação e Apresentação	Apresentação e demonstração prática	10%

Total: 100%.

15. Demonstração obrigatória de falha

Durante a apresentação final, cada grupo deverá demonstrar pelo menos uma situação de falha controlada. O grupo poderá, por exemplo, interromper um serviço, desconectar um cliente, simular indisponibilidade temporária ou provocar sobrecarga.

A demonstração deverá explicar: qual componente falhou; qual impacto era esperado; como o sistema detectou a falha; como reagiu; qual estado foi preservado; e como ocorreu a recuperação.

16. Regras de desenvolvimento e entrega

- O projeto deverá ser desenvolvido em grupo, com máximo de 6 integrantes.
- O código deverá estar versionado em um repositório GitHub, o qual deve obrigatoriamente ser compartilhado com o professor.
- Os commits deverão identificar adequadamente o trabalho realizado e o autor.
- O repositório deverá conter código, documentação e arquivos necessários para execução.
- A documentação deverá explicar arquitetura, tecnologias, protocolos, banco, consistência, tolerância a falhas e testes.
- Parte do acompanhamento e da avaliação da atividade serão feitas a partir das informações coletadas do repositório (código, documentação, commits, etc.).
- O sistema deverá ser executável para fins de avaliação.
- Funcionalidades que não possam ser executadas ou demonstradas poderão ser desconsideradas na avaliação.

17. Checklist para entrega final

☐ Código-fonte completo.

☐ README com instruções de execução.

☐ Diagrama de arquitetura.

☐ Home Broker funcional.

☐ Login/autenticação.

☐ Ativos e carteira.

☐ Envio de ordens.

☐ Livro de ofertas.

☐ Matching Engine.

☐ Atualização em tempo real.

☐ Persistência.

☐ Agentes/IA.

☐ Reconexão/tolerância a falhas.

☐ Testes de concorrência.

☐ Testes de carga.

☐ Logs/métricas.

☐ Demonstração de falha.

☐ Documentação técnica.

18. Observação pedagógica

O objetivo central do projeto não é construir apenas um aplicativo de compra e venda de ações. O domínio financeiro serve como cenário para a aplicação prática dos conceitos de Sistemas Distribuídos.

A avaliação considerará especialmente a capacidade do grupo de identificar problemas distribuídos, propor soluções, justificar decisões arquiteturais e demonstrar o comportamento do sistema sob concorrência, falhas e diferentes cargas.

Um sistema visualmente elaborado, mas que não demonstre adequadamente comunicação distribuída, sincronização, consistência, tolerância a falhas, escalabilidade e observabilidade, não atenderá plenamente aos objetivos do projeto.

19. Orientação sobre escopo

Os grupos não precisam implementar todas as características existentes em plataformas financeiras reais. O escopo deverá ser compatível com o tempo disponível e com o tamanho do grupo. A complexidade deverá estar principalmente nos desafios distribuídos e não apenas na quantidade de telas ou funcionalidades. Grupos que se destaquem ou surpreendam positivamente em seus projetos terão melhores avaliações.

Importante:

Será feita a **apresentação e entrega parcial do projeto no dia 21/10/2026.** A **apresentação e entrega final tem data prevista para o dia 30/11/2026**.

Observações:
Serão critérios de avaliação as funcionalidades implementadas, as tecnologias utilizadas, a facilidade no uso do programa, organização, qualidade da implementação, modularização, indentação do código, legibilidade e comentários no código. Os quesitos que não possam ser executados (sem compilar) serão desconsiderados durante a correção. Resolução com má legibilidade ou fora da especificação valerá no máximo 50% da pontuação da questão.

UNIVERSIDADE FEDERAL DE SERGIPE
DISCIPLINA DE SISTEMAS DISTRIBUÍDOS



Projeto de Sistemas Distribuídos

ATENÇÃO! Não utilize códigos de **outros humanos ou não-humanos**. A tentativa de plágio/cola será punida com nota zero (0,0) e com Procedimento Administrativo Disciplinar baseado na Resolução Nº 09/2016/CONEPE.